

24 Introduction to Bayesian Hierarchical Models

In a **Bayesian hierarchical**¹ model, the model for the data depends on certain parameters, and those parameters in turn depend on other parameters. Hierarchical modeling provides a framework for building complex and high-dimensional models from simple and low-dimensional building blocks.

Suppose we want to estimate the true, long run free throw percentage for a subset of NBA players². That is, we want to estimate each player's true probability of successfully scoring on any free throw attempt. We'll consider data from games in the 2022-2023 NBA season for players who were on the Philadelphia 76ers obtained from [basketball-reference](#) as of Feb 27, summarized in Section 24.1.1.

Example 24.1. Let θ_j denote player j 's true (long run) FT percentage; for example, θ_{Embiid} , θ_{Harden} , etc. Let n_j denote the number of free throw attempts and y_j the number of free throws made for player j .

1. One way to estimate θ_j is to pool all the data together. Over the 19 players in the data set, there are 1482 attempts, of which 1229 were successful, for an overall free throw percentage of 82.9%. The sample mean free throw percentage, among players with nonzero attempts, is 77.7%. So we could just estimate each θ_j with the overall free throw percentage (or the sample mean free throw percentage). What are some issues with this approach?
2. At the other extreme, we could estimate θ_j using just the attempts for player j . For example, use only the attempts for Embiid to estimate θ_{Embiid} , only the attempts for Harden to estimate θ_{Harden} , etc. What are some issues with this approach? (Hint: consider the players with no attempts, or few attempts.)

¹Hierarchical models are often called *multilevel*, *mixed effects*, or *random effects models*.

²Along with one bonus "player" who will illustrate some ideas.

3. In a Bayesian model, the θ_j 's would have a prior distribution. We could assume a Bayesian model with a prior in which all θ_j are independent. What are some issues with this approach?

Example 24.2. Let's first consider just a single player with true (long run) free throw percentage θ . Given n independent attempts, conditional on θ , we might assume that y follows a $\text{Binomial}(n, \theta)$ distribution.

1. If we imagine the player is randomly selected from among current NBA players, how might we interpret the prior distribution for θ ? Note: this question is mixing some Bayesian and frequentist ideas, but the purpose is to help intuition.
2. The prior distribution for θ has a prior mean; let's denote it μ . How might we interpret μ ?
3. The prior mean μ is also a parameter, so we could place a prior distribution on it. What would the prior distribution on μ represent?
4. The prior distribution for θ has a prior standard deviation; let's denote it σ . How might we interpret σ ?

5. The prior standard deviation σ is also a parameter, so we could place a prior distribution on it. What would the prior distribution on σ represent?
 6. In this model μ and θ are dependent. Why is this dependence useful? (Hint: consider another player.)
- A Bayesian hierarchical model consists of the following layers in the hierarchy.
 1. A model for the data y which depends on model parameters θ and determines the likelihood $f(y|\theta)$.
 - The model usually involves multiple parameters. Generically, θ represents a vector of parameters that show up in the likelihood function.
 - Remember: While the likelihood for a particular y sample is computed by evaluating the conditional density of y given θ , the likelihood function is *a function of θ , treating the data y as fixed.*
 2. A (joint) prior distribution for the model parameters θ , denoted $\pi(\theta|\phi)$, which depends on *hyperparameters* ϕ .
 3. A *hyperprior* distribution, i.e., a prior distribution for the hyperparameters, denoted $\pi(\phi)$. (Again, ϕ generically represents a vector of hyperparameters.)
 - The above specification implicitly assumes that the data distribution and likelihood function only depend on θ ; the hyperparameters only affect y through θ . That is, the general hierarchical model above assumes that the data y and hyperparameters ϕ are conditionally independent given the model parameters θ , so the likelihood satisfies

$$f(y|\theta, \phi) = f(y|\theta)$$

The joint prior distribution on parameters and hyperparameters is

$$\pi(\theta, \phi) = \pi(\theta|\phi)\pi(\phi)$$

- Given the observed data y , we find the joint posterior density of the parameters and hyperparameters, denoted $\pi(\theta, \phi|y)$, using Bayes rule: posterior $\propto$ likelihood $\times$ prior.

$$\begin{aligned}\pi(\theta, \phi|y) &\propto f(y|\theta, \phi)\pi(\theta, \phi) \\ &\propto f(y|\theta)\pi(\theta|\phi)\pi(\phi)\end{aligned}$$

- In Bayesian hierarchical models, there will generally be implied dependence between parameters. But dependence among parameters is actually useful for several reasons:
 - The dependence is natural and meaningful in the problem context.
 - If parameters are dependent, then all data can inform parameter estimates.

Example 24.3. Now suppose want to estimate the free throw percentages for J NBA players, given data consisting of the number of free throws attempted n_j and made y_j by player j . Specify a hierarchical model, including your choice of prior distribution. How many parameters are there in the model?

Example 24.4. Use software to fit the Bayesian model using the 76ers data. Summarize the posterior distribution of μ .

Example 24.5. For each player j , consider the posterior mean and posterior SD of θ_j .

1. What is the posterior mean of θ_j for players with no attempts? Why does this make sense?

2. Why is the posterior mean of θ_j not the same for players with no attempts?
3. For which players is the posterior mean closest to the observed free throw percentage? Why?
4. Which players have the smallest posterior SDs? Why?
5. Find and interpret 50%, 80%, and 98% posterior credible intervals for Joel Embiid.
6. Find and interpret 50%, 80%, and 98% posterior credible intervals for Kevin Ross.

24.1 Notes

24.1.1 Sample data

Player	FTA	FT	FTpct
Joel Embiid	556	477	0.858
James Harden	279	241	0.864
Tyrese Maxey	155	126	0.813
Tobias Harris	90	77	0.856
Shake Milton	89	76	0.854
Montrezl Harrell	85	59	0.694
De'Anthony Melton	59	46	0.780
Danuel House Jr.	40	28	0.700
Georges Niang	32	28	0.875
Furkan Korkmaz	29	19	0.655
Paul Reed	22	16	0.727
P.J. Tucker	20	17	0.850
Matisse Thybulle	12	9	0.750
Jalen McDaniels	5	2	0.400
Kevin Ross	5	5	1.000
Jaden Springer	4	3	0.750
Saben Lee	0	0	NA
Julian Champagnie	0	0	NA
Michael Foster Jr.	0	0	NA

```
data = read_csv("sixers.csv") |>
  rename(FTpct = "FT%") |>
  arrange(desc(FTA))
```

```
data |>
  select(Player, FTA, FT, FTpct) |>
  kbl() |>
  kable_styling()
```

24.1.2 Approximating the posterior distribution

brms is structured around linear models. In the situation of this example, **brms** wants to fit a logistic regression model. Rather than doing that, I used another software program, JAGS, to fit our current model. Don't worry about the syntax. Just notice that the inputs are: (1) the data, (2) the model for the data which determines the likelihood (**dbinom**), and (3) the prior (**dbeta**) and hyperprior (**dnorm**). And the output is an approximation of the posterior distribution of the parameters.

```
library(rjags)
```

Loading required package: coda

Linked to JAGS 4.3.2

Loaded modules: basemod,bugs

```
player = data$Player
y = data$FT
n = data$FTA
J = length(y)

# specify the model: likelihood, prior, hyperprior

model_string <- "model{

  # Likelihood
  for (j in 1:J){
    y[j] ~ dbinom(theta[j], n[j])
  }

  # Prior Beta(alpha, beta)
  # We put hyperpriors on the mean mu and the SD sigma,
  # So we need to convert to shape parameters alpha and beta

  for (j in 1:J){
    theta[j] ~ dbeta(mu * (mu * (1 - mu) / sigma ^ 2 - 1), (1 - mu) * (mu * (1 - mu) / sigma
  }

  # Hyperprior, phi=(mu, sigma)
  # In JAGS the second parameter of dnorm is precision = 1 / SD ^ 2

  mu ~ dnorm(0.75, 1 / 0.05 ^ 2)
  sigma ~ dnorm(0.08, 1 / 0.02 ^ 2)
}"

dataList = list(y = y, n = n, J = J)

model <- jags.model(file = textConnection(model_string), data = dataList)
```

```

Compiling model graph
  Resolving undeclared variables
  Allocating nodes
Graph information:
  Observed stochastic nodes: 19
  Unobserved stochastic nodes: 21
  Total graph size: 77

```

```

Initializing model

```

```

update(model, n.iter = 1000)

posterior_sample <- coda.samples(
  model,
  variable.names = c("theta", "mu", "sigma"),
  n.iter = 10000,
  n.chains = 5
)

```

```

head(posterior_sample)

```

```

[[1]]
Markov Chain Monte Carlo (MCMC) output:
Start = 2001
End = 2007
Thinning interval = 1
      mu      sigma theta[1] theta[2] theta[3] theta[4] theta[5]
[1,] 0.7992296 0.04808789 0.8444957 0.8642311 0.8532882 0.8434687 0.8172154
[2,] 0.7895968 0.05257005 0.8472829 0.8271506 0.8638483 0.8460952 0.8170892
[3,] 0.7777101 0.04551532 0.8484874 0.8283909 0.8410142 0.8256072 0.8116226
[4,] 0.7863290 0.03995309 0.8554542 0.8413460 0.7874767 0.8220231 0.8125987
[5,] 0.7839165 0.06423043 0.8531644 0.8420853 0.8283274 0.8081782 0.8455710
[6,] 0.7800473 0.06555810 0.8295799 0.8480622 0.8206632 0.8424045 0.8332588
[7,] 0.7806023 0.09813775 0.8311649 0.8559649 0.7997031 0.8182367 0.8615528
      theta[6] theta[7] theta[8] theta[9] theta[10] theta[11] theta[12]
[1,] 0.7743131 0.8071986 0.7093785 0.8099872 0.7480232 0.7335395 0.8730629
[2,] 0.7054682 0.8026152 0.7517129 0.8242733 0.7385338 0.8197124 0.8613113
[3,] 0.6933574 0.7875759 0.7581551 0.8235767 0.7805816 0.7466106 0.8152298
[4,] 0.7013122 0.8091125 0.7792442 0.8169829 0.7241672 0.7591444 0.8349705
[5,] 0.7733447 0.7879380 0.7240846 0.7201947 0.7412328 0.7505033 0.8300862
[6,] 0.7732933 0.7870132 0.6923770 0.7424927 0.7827225 0.7604972 0.8310564
[7,] 0.7513459 0.6821973 0.7669859 0.8825732 0.6659067 0.7628825 0.8198409

```

```

      theta[13] theta[14] theta[15] theta[16] theta[17] theta[18] theta[19]
[1,] 0.7891976 0.7586213 0.8390844 0.8397455 0.8695535 0.8139644 0.7549827
[2,] 0.7875408 0.7938380 0.8345334 0.7312227 0.7919161 0.7969242 0.7563891
[3,] 0.7481139 0.7113666 0.7971687 0.7943850 0.8163589 0.8172472 0.7444062
[4,] 0.8326740 0.7253460 0.8102735 0.8347320 0.7793726 0.7511220 0.6833952
[5,] 0.8177928 0.8063942 0.8398323 0.7417911 0.7877453 0.7658758 0.6922788
[6,] 0.7707731 0.6430727 0.8149007 0.7644496 0.8089858 0.7928053 0.5748382
[7,] 0.7631681 0.6537589 0.8683869 0.8097285 0.8272409 0.7278358 0.5364067

```

```

attr("class")
[1] "mcmc.list"

```

```
summary(posterior_sample)
```

```

Iterations = 2001:12000
Thinning interval = 1
Number of chains = 1
Sample size per chain = 10000

```

1. Empirical mean and standard deviation for each variable,
plus standard error of the mean:

	Mean	SD	Naive SE	Time-series SE
mu	0.78304	0.02229	0.0002229	0.0005386
sigma	0.07358	0.01593	0.0001593	0.0004171
theta[1]	0.85361	0.01457	0.0001457	0.0001908
theta[2]	0.85559	0.02043	0.0002043	0.0002682
theta[3]	0.80774	0.02911	0.0002911	0.0003793
theta[4]	0.83648	0.03422	0.0003422	0.0004501
theta[5]	0.83562	0.03428	0.0003428	0.0004726
theta[6]	0.71899	0.04297	0.0004297	0.0006237
theta[7]	0.78174	0.04338	0.0004338	0.0005908
theta[8]	0.73724	0.05397	0.0005397	0.0008319
theta[9]	0.83031	0.04729	0.0004729	0.0007353
theta[10]	0.72105	0.06080	0.0006080	0.0009529
theta[11]	0.76052	0.06114	0.0006114	0.0008431
theta[12]	0.81085	0.05607	0.0005607	0.0008125
theta[13]	0.77403	0.06637	0.0006637	0.0010378
theta[14]	0.72760	0.08119	0.0008119	0.0013970
theta[15]	0.81476	0.06762	0.0006762	0.0010580
theta[16]	0.77993	0.07315	0.0007315	0.0011842

theta[17]	0.78236	0.07952	0.0007952	0.0013073
theta[18]	0.78344	0.07693	0.0007693	0.0012953
theta[19]	0.78379	0.07680	0.0007680	0.0012459

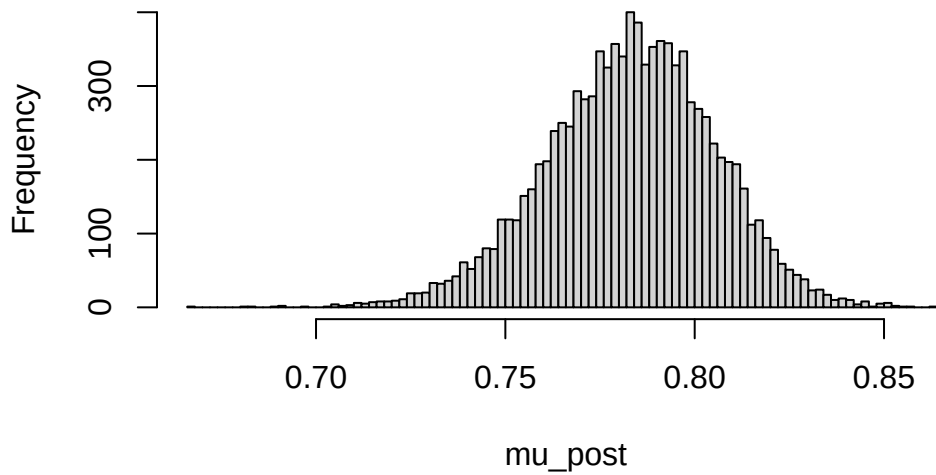
2. Quantiles for each variable:

	2.5%	25%	50%	75%	97.5%
mu	0.73690	0.76863	0.78384	0.79801	0.8243
sigma	0.04394	0.06272	0.07306	0.08375	0.1059
theta[1]	0.82406	0.84396	0.85388	0.86379	0.8814
theta[2]	0.81300	0.84235	0.85625	0.86971	0.8933
theta[3]	0.74672	0.78876	0.80892	0.82788	0.8608
theta[4]	0.76520	0.81439	0.83826	0.86015	0.8985
theta[5]	0.76232	0.81359	0.83695	0.85979	0.8974
theta[6]	0.62997	0.69068	0.72102	0.74871	0.7985
theta[7]	0.68989	0.75408	0.78380	0.81211	0.8602
theta[8]	0.62270	0.70224	0.74032	0.77536	0.8337
theta[9]	0.72963	0.80040	0.83317	0.86314	0.9148
theta[10]	0.59234	0.68238	0.72513	0.76419	0.8288
theta[11]	0.62641	0.72295	0.76527	0.80351	0.8672
theta[12]	0.69124	0.77563	0.81426	0.85087	0.9102
theta[13]	0.62878	0.73303	0.77977	0.82090	0.8891
theta[14]	0.54973	0.67766	0.73521	0.78520	0.8661
theta[15]	0.66656	0.77281	0.81913	0.86204	0.9330
theta[16]	0.61871	0.73557	0.78667	0.83085	0.9069
theta[17]	0.60564	0.73430	0.78926	0.83797	0.9180
theta[18]	0.60949	0.73776	0.79002	0.83686	0.9152
theta[19]	0.61529	0.73680	0.78940	0.83740	0.9159

24.1.3 Posterior distribution of μ

```
mu_post = as.matrix(posterior_sample)[, "mu"]
hist(mu_post, breaks = 100)
```

Histogram of mu_post



```
quantile(mu_post, c(0.01, 0.10, 0.25, 0.5, 0.75, 0.90, 0.99))
```

1%	10%	25%	50%	75%	90%	99%
0.7270011	0.7544668	0.7686256	0.7838388	0.7980098	0.8107315	0.8322121

24.1.4 Posterior mean and SD of θ_j s

```
player_summary = data.frame(player, n, y, y / n)
names(player_summary) = c("player", "FTA", "FT", "FTpct")

# In posterior_sample list, the first two entries correspond to mu and sigma,
# so we remove them.

player_summary[c("posterior_mean", "posterior_SD")] =
  summary(posterior_sample)$statistics[-(1:2), c("Mean", "SD")]

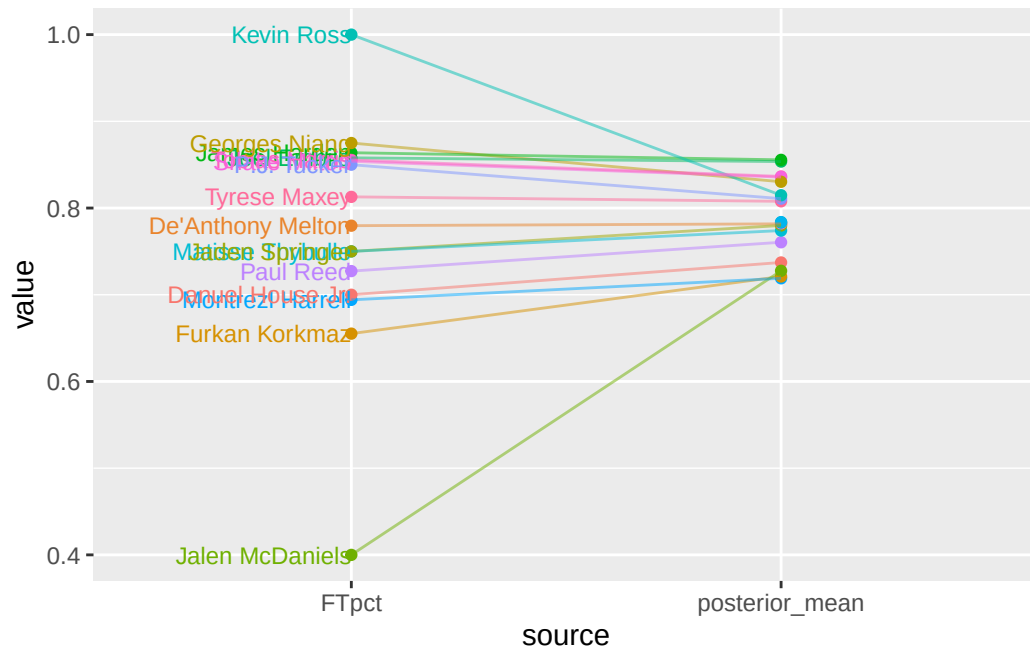
player_summary |> kbl(digits = 4) |> kable_styling()
```

player	FTA	FT	FTpct	posterior_mean	posterior_SD
Joel Embiid	556	477	0.8579	0.8536	0.0146
James Harden	279	241	0.8638	0.8556	0.0204
Tyrese Maxey	155	126	0.8129	0.8077	0.0291
Tobias Harris	90	77	0.8556	0.8365	0.0342
Shake Milton	89	76	0.8539	0.8356	0.0343
Montrezl Harrell	85	59	0.6941	0.7190	0.0430
De'Anthony Melton	59	46	0.7797	0.7817	0.0434
Danuel House Jr.	40	28	0.7000	0.7372	0.0540
Georges Niang	32	28	0.8750	0.8303	0.0473
Furkan Korkmaz	29	19	0.6552	0.7211	0.0608
Paul Reed	22	16	0.7273	0.7605	0.0611
P.J. Tucker	20	17	0.8500	0.8109	0.0561
Matisse Thybulle	12	9	0.7500	0.7740	0.0664
Jalen McDaniels	5	2	0.4000	0.7276	0.0812
Kevin Ross	5	5	1.0000	0.8148	0.0676
Jaden Springer	4	3	0.7500	0.7799	0.0732
Saben Lee	0	0	NaN	0.7824	0.0795
Julian Champagne	0	0	NaN	0.7834	0.0769
Michael Foster Jr.	0	0	NaN	0.7838	0.0768

```

player_summary |>
  select(player, FTpct, posterior_mean) |>
  pivot_longer(!player, names_to = "source", values_to = "value") |>
  mutate(label = if_else(source == "FTpct", player, "")) |>
  ggplot(aes(
    x = source,
    y = value,
    group = player,
    col = player,
    label = label
  )) +
  scale_color_discrete() +
  geom_point() +
  geom_text(hjust = 1, size = 3) +
  geom_line(alpha = 0.5) +
  theme(legend.position = "none")

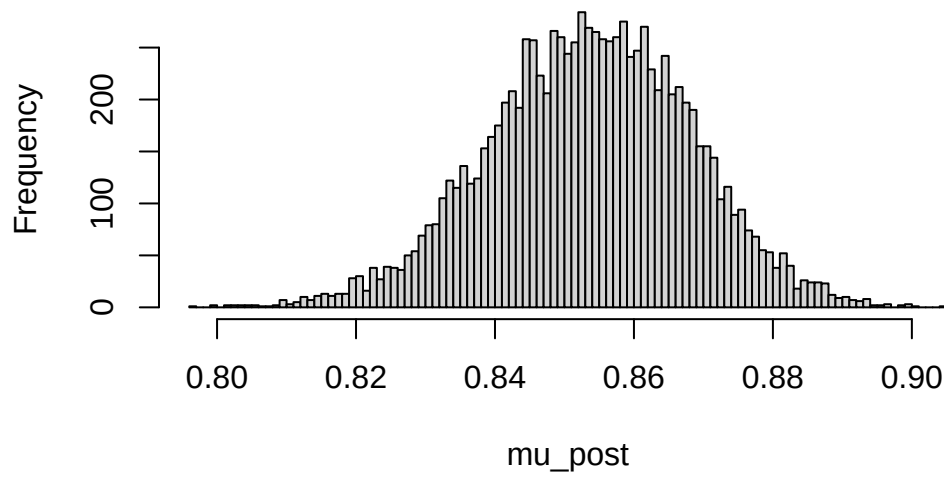
```



24.1.5 Posterior distribution of θ_{Embiid}

```
mu_post = as.matrix(posterior_sample)[, "theta[1]"]
hist(mu_post, breaks = 100)
```

Histogram of mu_post



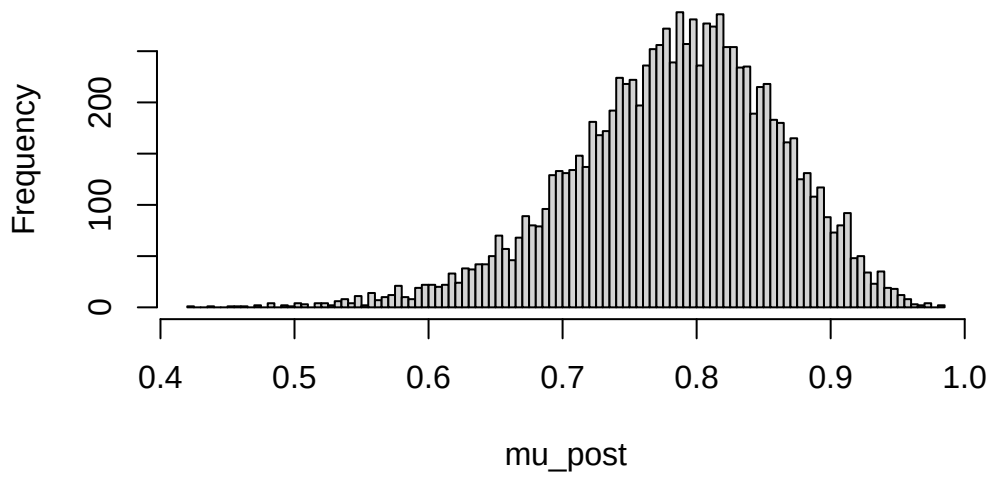
```
quantile(mu_post, c(0.01, 0.10, 0.25, 0.5, 0.75, 0.90, 0.99))
```

1%	10%	25%	50%	75%	90%	99%
0.8182763	0.8347282	0.8439649	0.8538827	0.8637909	0.8718126	0.8865613

24.1.6 Posterior distribution of θ_{Ross}

```
mu_post = as.matrix(posterior_sample)[, "theta[19]"]  
  
hist(mu_post, breaks = 100)
```

Histogram of mu_post



```
quantile(mu_post, c(0.01, 0.10, 0.25, 0.5, 0.75, 0.90, 0.99))
```

1%	10%	25%	50%	75%	90%	99%
0.5733430	0.6848579	0.7368022	0.7894037	0.8374026	0.8776946	0.9352280

[ShinyStan](#) provides a nice interface to interactively investigate the posterior distribution.

```
library(shinystan)

sixers_sso <- as.shinystan(posterior_sample, model_name = "76ers Free Throws")
sixers_sso <- drop_parameters(sixers_sso, pars = c("sigma"))
launch_shinystan(sixers_sso)
```